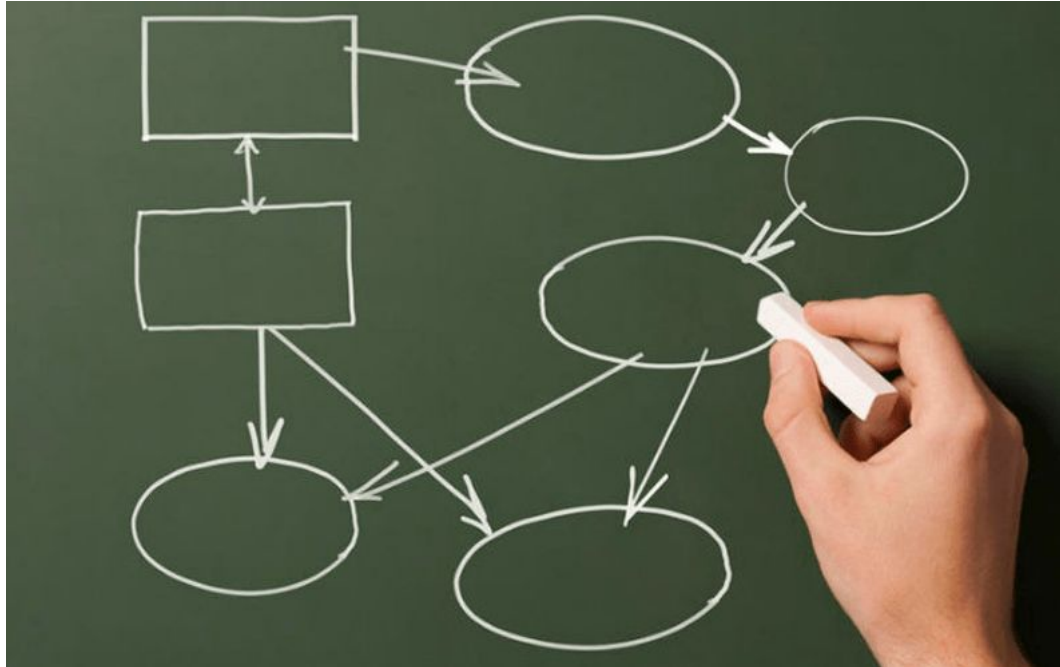
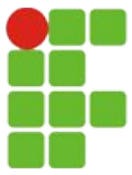
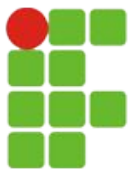




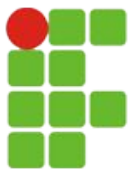
Algoritmos e programação– ALG

Prof. Jones

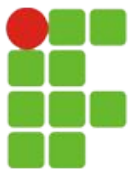


Últimas aulas ...

- ✓ Repetição Condicional **Faça Até (pós-teste)**
- ✓ Repetição Condicional **Enquanto Faça (pré-teste)**
- ✓ Repetição Contada Para Até Faça (iterativo)
- ✓ Repetição SELETIVA



Arquivos

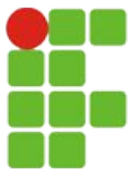


Introdução

- ✓ **As estruturas vistas anteriormente armazenam as informações na memória principal do computador.**
- ✓ **Nem sempre é conveniente.**

Problemas:

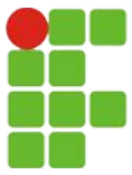
- ✓ **A informação é perdida;**
- ✓ **As estruturas de dados são limitadas;**
- ✓ **Existe uma quantidade de informação que pode ser armazenada para resolver o problema.**



Introdução

- ✓ Algumas informações geradas pelos programas precisam ser mantidas para posterior uso
- ✓ Exs: **cadastro de alunos, cadastro de vendas de uma empresa, estatísticas ...**





Arquivos

- Um arquivo é armazenado em um dispositivo de memória secundária. Pode ser lido e escrito por um programa.
- Em programação, existem vários tipos de informações que podem ser armazenadas em arquivo.

Exemplos:

Arquivo de
inteiros

1
56
0
23

Arquivo de
reais

21.5
-2.67
3.75
0.6

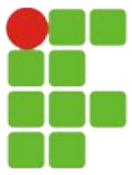
Arquivo de
vetores de
tamanho 4
composto
por
inteiros

(1, 23, -12, 4)
(56, 0, 65, 2)
(0, 41, -7, 60)

Arquivo de
cadeia de
caracteres

Casa
Lua
Giz
Rio

Representação Simbólica



Arquivos

Na linguagem C++

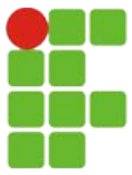
- Podem ser criados arquivos:

- “modo texto”
- “modo binário”

Ainda que eu falasse\n
a língua dos homens\n
Que eu falasse\n
a língua dos anjos\n
Sem amor eu nada seria\n
EOF

- ❖ **Modo texto:** possui caractere de nova linha e de fim de arquivo
- ❖ **Modo binário:** informações armazenadas na forma de bytes, sem marca de final de arquivo ou final de linha.

0000	FF D8 FF E1	1D FE 45 78	69 66 00 00	49 49 2A 00
0010	08 00 00 00	09 00 0F 01	02 00 06 00	00 00 7A 00
0020	00 00 10 01	02 00 14 00	00 00 80 00	00 00 12 01
0030	03 00 01 00	00 00 01 00	00 00 1A 01	05 00 01 00
0040	00 00 A0 00	00 00 1B 01	05 00 01 00	00 00 A8 00
0050	00 00 28 01	03 00 01 00	00 00 02 00	00 00 32 01
0060	02 00 14 00	00 00 B0 00	00 00 13 02	03 00 01 00
0070	00 00 01 00	00 00 69 87	04 00 01 00	00 00 C4 00
0080	00 00 3A 06	00 00 43 61	6E 6F 6E 00	43 61 6E 6F
0090	6E 20 50 6F	77 65 72 53	68 6F 74 20	41 36 30 00
00A0	00 00 00 00	00 00 00 00	00 00 00 00	B4 00 00 00
00B0	01 00 00 00	B4 00 00 00	01 00 00 00	32 30 30 34
00C0	3A 30 36 3A	32 35 20 31	32 3A 33 30	3A 32 35 00
00D0	1F 00 9A 82	05 00 01 00	00 00 86 03	00 00 9D 82
00E0	05 00 01 00	00 00 8E 03	00 00 00 90	07 00 04 00



Funções em C++ (**fstream**)

open(): Esta função ajuda a criar um arquivo e abrir o arquivo em diferentes modos, como operações de entrada, operações de saída, modo binário, etc.

close(): Esta função ajuda a fechar um arquivo existente.

in: Prepara para ler um arquivo.

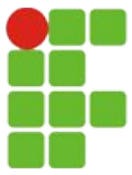
out: Prepara para escrever em um arquivo.

app: Prepara para editar em um arquivo existente com dados.

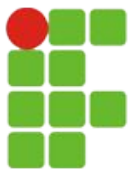
eof(): Avalia o fim do arquivo.

>> : Ajuda a ler dados de um arquivo.

<< : Ajuda a escrever dados em um arquivo.



Criando um arquivo novo no disco



Criando um arquivo novo de texto

Incluir a biblioteca de manipulação de arquivos:

```
#include<fstream>
```

Declarar uma variável do tipo (**ofstream**)

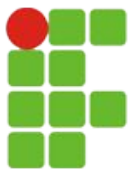

output (saída) **file** (arquivo)

```
ofstream Arquivo;
```

Cria/abre o arquivo seguindo a sintaxe: **Arquivo.open(nome_arquivo,modo_abertura);**

```
Arquivo.open("meuarquivo.txt", ios::out);
```

Este comando vai criar o arquivo "meuarquivo.txt" **SE** ele não existir, ou se ele existir abrir e apagar **TODOS OS DADOS**.



Criando um arquivo novo de texto

Ler o dado do teclado

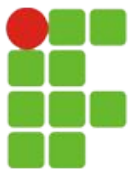
```
string nome;  
getline(cin, nome);
```

Escrever o dado no arquivo

```
Arquivo << nome << endl;
```

Fechar o arquivo

```
Arquivo.close();
```

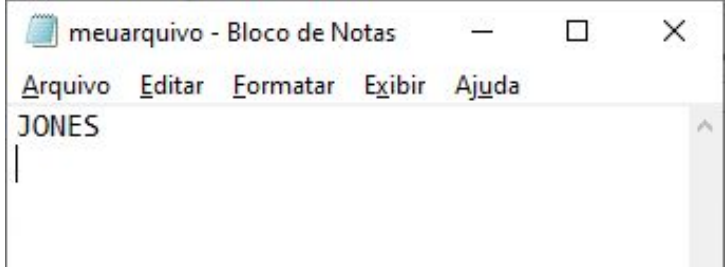


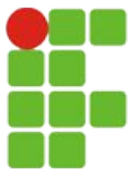
Criando um arquivo novo de texto

```
#include<iostream>
#include<fstream>
#include<string>

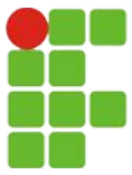
using namespace std;

int main()
{
    string nome;
    ofstream Arquivo;
    Arquivo.open("meuarquivo.txt", ios::out);
    if(! Arquivo){
        cout << "O arquivo nao pode ser aberto" << endl;
    }
    else
    {
        cout << "Informe o seu nome e aperte a tecla ENTER" << endl;
        getline(cin, nome);
        Arquivo << nome << endl;
        Arquivo.close();
        cout << "Arquivo gravado no disco!" << endl;
    }
    return 0;
}
```





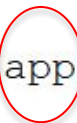
Editando um arquivo existente

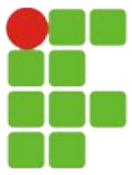


```
#include<iostream>
#include<fstream>
#include<string>

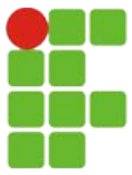
using namespace std;

int main()
{
    string nome;
    ofstream Arquivo;
    Arquivo.open("meuarquivo.txt", ios::app);
    if(! Arquivo){
        |   cout << "O arquivo nao pode ser aberto" << endl;
    }
    else
    {
        cout << "Informe o seu nome e aperte a tecla ENTER" << endl;
        getline(cin, nome);
        Arquivo << nome << endl;
        Arquivo.close();
        cout << "Arquivo gravado no disco!" << endl;
    }
    return 0;
}
```





Ler um arquivo existente no disco



Ler um arquivo

Incluir a biblioteca de manipulação de arquivos:

```
#include<fstream>
```

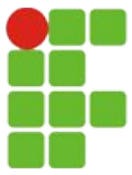
Declarar uma variável do tipo (**ifstream**)


input (saída) **file** (arquivo)

```
ifstream Arquivo;
```

Abre o arquivo seguindo a sintaxe: **Arquivo.open(nome_arquivo,modo_abertura);**

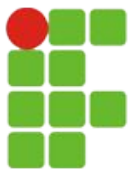
```
Arquivo.open("meuarquivo.txt", ios::in);
```

Ler um arquivo

Repetição enquanto não encontrar o final do arquivo

```
while ( ! Arquivo.eof())  
{  
    getline (Arquivo,linha); ← Pega a linha do arquivo e joga na variável linha  
    cout << linha << endl; ← Mostra a variável linha  
}
```



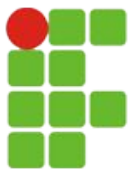
Ler um arquivo

```
#include<iostream>
#include<fstream>
#include<string>

using namespace std;

int main()
{
    string linha;
    ifstream Arquivo;
    Arquivo.open("meuarquivo.txt", ios::in);
    while ( ! Arquivo.eof())
    {
        getline (Arquivo, linha);
        cout << linha << endl;
    }
    Arquivo.close();
    return 0;
}
```





- 1) Faça um programa que irá funcionar como uma agenda, contendo nome, telefone e e-mail de cada contato. O programa deve possuir funções para criar a agenda, permitir adicionar novos nomes e fazer buscas de dois tipos:
 - a) mostrar a lista toda de nomes
 - b) buscar se um nome existe na lista.

2) Faça um programa que gera um arquivo contendo uma lista de cidades e suas temperaturas, como ilustrado abaixo. O programa deve possuir rotinas para criar o arquivo e para ler os dados do arquivo. Faça uma rotina que calcula a temperatura média das cidades registradas no arquivo e mostra quais cidades possuem temperatura acima da média.

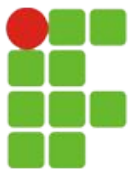
•Exemplo:

São Carlos 23.2
Piracicaba 25.4
São Paulo 20.8
Curitiba 17.5

saída da função:

média = 21.7

acima da média: São Carlos e Piracicaba



3) Você foi convidado a desenvolver uma rotina de comparação de respostas do sistema da Maratona de programação (BOCA). O BOCA utiliza um arquivo de gabarito fornecido pelos juízes que elaboraram a prova, e gera um arquivo de saída do programa enviado pela equipe. Crie um arquivo “**gabarito.txt**” e coloque as saídas dos juízes, em seguida crie um arquivo simulando a da equipe, compare se os dois arquivos são iguais:

caso SIM mostrar **YES**

caso tenha algum caractere diferente mostrar **NO – Wrong Answer**

caso tenha equipe uma quebra de linha no final **NO – Presentation Error**

Utilize **string compare!**

Gabarito do Juiz

1
2
3
4
5

Gabarito da equipe

1
2
6
9
4

